

Transformers: একটি সম্পূর্ণ টেকনিক্যাল গাইড

Transformer Architecture থেকে শুরু করে তার Origin Story, Impact, Advantages, Disadvantages এবং ভবিষ্যৎ — সবকিছু বাংলায় বিস্তারিতভাবে।

● "Attention is All You Need" (2017)
অবলম্বনে

● সম্পূর্ণ বাংলা
ব্যাখ্যা

● Python কোড
সহ

📋 বিষয়বস্তুর তালিকা

01 Transformer কী? — সহজ পরিচয়

02 Transformer-এর গভীর প্রভাব (Impact)

03 Origin Story — তিনটি গুরুত্বপূর্ণ Paper

04 সম্পূর্ণ Timeline (২০০০-২০২৩)

05 Architecture Overview

06 Advantages — কেন Transformer সেরা?

07 Disadvantages — কী কী সীমাবদ্ধতা?

08 বিখ্যাত Applications

09 ভবিষ্যৎ — কোথায় যাচ্ছে Transformer?

10 Python কোড উদাহরণ

আমরা এই কোর্সে এর আগে তিনটি Neural Network Architecture পড়েছি — **ANN (Artificial Neural Network)**, **CNN (Convolutional Neural Network)** এবং **RNN (Recurrent Neural Network)**। প্রতিটি Architecture ভিন্ন ধরনের data-এর জন্য তৈরি।



ANN (Artificial Neural Network)

Tabular data-এর জন্য সবচেয়ে ভালো কাজ করে।
যেমন স্প্রেডশিটের row ও column-এর data।



CNN (Convolutional Neural Network)

Image-based data-এর জন্য তৈরি। ছবি চেনা, object detection ইত্যাদিতে ব্যবহার হয়।



RNN (Recurrent Neural Network)

Sequential data-এর জন্য, যেমন text বা speech — যেখানে ক্রম (order) গুরুত্বপূর্ণ।



Transformer

Sequence-to-Sequence task handle করার জন্য তৈরি। RNN-এর চেয়ে অনেক বেশি শক্তিশালী ও দ্রুত।



সংজ্ঞা

Transformer হলো একটি **Neural Network Architecture** যা বিশেষভাবে Sequence-to-Sequence task handle করার জন্য তৈরি। এর নাম "Transformer" কারণ এটি এক sequence-কে আরেক sequence-এ *transform* করে।

| Sequence-to-Sequence Task কী?

Sequence-to-Sequence (Seq2Seq) task হলো এমন একটি কাজ যেখানে **input-ও sequential data** এবং **output-ও sequential data**। উদাহরণ হিসেবে:



Machine Translation



Question Answering System

ইংরেজি বাক্যকে বাংলায় অনুবাদ করা। Input: English sentence → Output: Bengali sentence।

একটি প্রশ্নের উত্তর দেওয়া। Input: Question → Output: Answer।



Text Summarization

বড় text-এর সারসংক্ষেপ তৈরি। Input: Long text → Output: Summary।

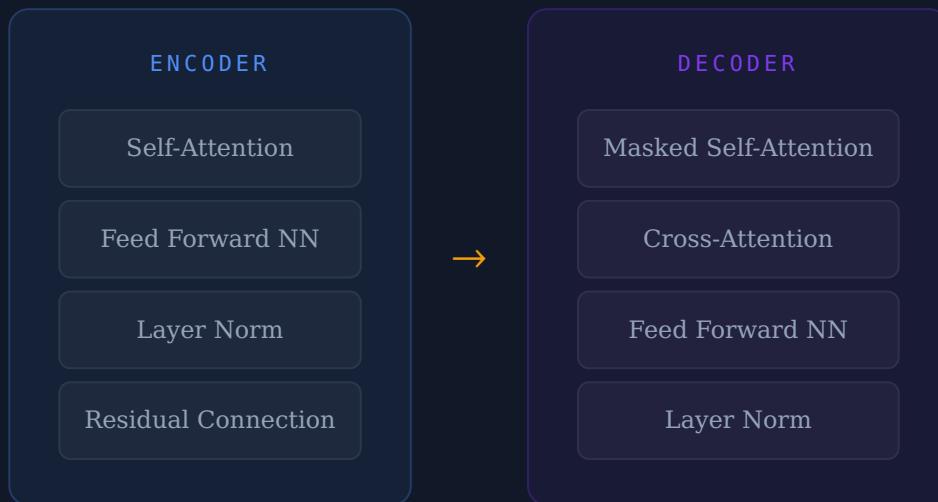


Chatbot Conversation

User-এর বার্তার জবাব দেওয়া। Input: User message → Output: Bot response।

Transformer-এর মূল বৈশিষ্ট্য (High-Level Overview)

Transformer Architecture-এর সরলীকৃত দৃশ্য:



পূর্ববর্তী Seq2Seq Architecture-এর মতো এখানেও Encoder ও Decoder আছে — কিন্তু LSTM-এর পরিবর্তে **Self-Attention** ব্যবহার হয়।



মূল পার্থক্য

পূর্ববর্তী Seq2Seq Architecture-এ **LSTM** ব্যবহার হতো, কিন্তু Transformer-এ **Self-Attention** ব্যবহার হয়। এই Self-Attention-এর কারণে Encoder একসাথে পুরো sentence-এর সব word Parallel-ভাবে process করতে পারে — ফলে training অনেক দ্রুত ও scalable হয়।

Transformer-এর গভীর প্রভাব (Impact)

Transformer শুধু একটি Architecture নয় — এটি পুরো AI ও Deep Learning জগৎকে বদলে দিয়েছে। ২০১৭ সালে "Attention is All You Need" paper-এ প্রথম প্রকাশিত হওয়ার পর থেকে এর প্রভাব অভূতপূর্ব। ChatGPT, আজকের বিশ্বের সবচেয়ে জনপ্রিয় software, মূলত একটি Transformer।

01

NLP ফিল্ডের বিপ্লব

Transformer-এর জন্মই হয়েছিল NLP (Natural Language Processing) সমস্যা সমাধানের জন্য। Machine Translation, Sentiment Analysis, Question Answering — সব ক্ষেত্রেই Transformer State-of-the-Art result দিচ্ছে। যে progress ৫০ বছরে হওয়ার কথা ছিল, তা মাত্র ৫ বছরে হয়ে গেছে।

02

AI-কে Democratize করা

আগে NLP application বানাতে scratch থেকে model train করতে হতো — প্রচুর data, টাকা ও সময় লাগত। এখন BERT বা GPT-এর মতো pre-trained Transformer download করে নিজের কাজে Fine-tune করা যায়। HuggingFace-এর মতো library মাত্র ৩-৪ লাইনের কোডে এই কাজ করে দেয়।

03

Multimodal Capability

Transformer শুধু text নয় — images, speech, video, sensor data সব ধরনের data নিয়ে কাজ করতে পারে। Input হিসেবে image দিলে text output পাওয়া যায়, আবার text দিলে image তৈরি করা যায় (যেমন DALL-E, Midjourney)। এই flexibility-ই Transformer-কে অনন্য করে তুলেছে।

04

Generative AI-এর ত্বরণ

Generative AI-এর সবচেয়ে বড় jump Transformer-এর কারণে হয়েছে। ChatGPT (text generation), DALL-E (image generation), Runway ML (video generation) — সব Transformer-based। এই ক্ষেত্রে startup-এর বিস্ফোরণ ও কোটি কোটি ডলারের investment Transformer-এরই ফল।

Deep Learning-এর Unification

আগে ভিন্ন ভিন্ন সমস্যার জন্য ভিন্ন ভিন্ন Architecture লাগত — tabular data-তে ANN, image-এ CNN, sequence-এ RNN। এখন NLP, Computer Vision, Reinforcement Learning, Science Research — সব ক্ষেত্রেই Transformer ব্যবহার হচ্ছে। এটি Deep Learning-এর Unification।

Transformer-এর জন্ম হঠাৎ হয়নি। এর পেছনে রয়েছে তিনটি landmark research paper-এর ক্রমবর্ধমান উন্নতির ইতিহাস।

Chapter 1: Seq2Seq Architecture (2014)

PAPER

"Sequence to Sequence Learning with Neural Networks" — Elias Sutskever et al. (2014)

এই paper-এ Machine Translation-এর জন্য Encoder-Decoder Architecture প্রস্তাব করা হয়। উভয় Encoder ও Decoder-এ LSTM ব্যবহার হয়।



ANALOGY — সহজ উদাহরণ

মনে করো তুমি একটি বই পড়ছ এবং সেটার সারসংক্ষেপ একটি notecard-এ লিখছ। Encoder হলো তোমার পড়ার প্রক্রিয়া, আর Context Vector হলো সেই notecard। Decoder সেই notecard দেখে অনুবাদ করে। কিন্তু বইটি যদি অনেক বড় হয়, তাহলে একটি ছোট notecard-এ সব ধরা সম্ভব নয়!

কিভাবে কাজ করত:



Input: "My name is Nitesh" (ইংরেজি)

Encoder LSTM প্রতিটি word step-by-step process করে:

Input Sentence: [My] → [name] → [is] → [Nitesh]

↓ ↓ ↓ ↓

LSTM States: h1 → h2 → h3 → h4

↓

Context Vector (CV)

↓

Decoder LSTM: CV → [মেরা] → [নাম] → [নিতেশ] → [হায়]

সমস্যা: Sentence বড় হলে Context Vector সব তথ্য ধরতে পারে না!

"My name is Nitesh and I love machine learning and coding..."

→ ৩০+ words হলে Context Vector inadequate হয়ে পড়ে



সমস্যা

Sentence দীর্ঘ হলে (৩০+ words) এই architecture fail করে। পুরো sentence-কে একটি fixed-size Context Vector-এ compress করা সম্ভব হয় না — ফলে translation quality খারাপ হয়।

Chapter 2: Attention Mechanism (2015)



PAPER

"Neural Machine Translation by Jointly Learning to Align and Translate" — Bahdanau et al. (2015)

এই paper-ই প্রথম **Attention Mechanism**-এর ধারণা দেয় — পূর্ববর্তী paper-এর Context Vector সমস্যা সমাধানের জন্য।



ANALOGY — ATTENTION কীভাবে কাজ করে

একটি দোভাষীর কথা চিন্তা করো। সে পুরো বইটা মুখস্থ না করে, যখন একটি নির্দিষ্ট শব্দ অনুবাদ করে, তখন সে বইয়ের সেই অংশটিই আবার দেখে যেটি সেই শব্দের জন্য প্রাসঙ্গিক। "Turn off the lights" → "লাইট" অনুবাদ করতে শুধু "lights" শব্দটার দিকে মনোযোগ দেওয়া দরকার।



ATTENTION MECHANISM – CONCEPTUAL FLOW

Input: "Turn off the lights" → Output: হিন্দিতে অনুবাদ

Encoder Hidden States:

```
h1 = encode("Turn")
h2 = encode("off")
h3 = encode("the")
h4 = encode("lights")
```

Decoder যখন "লাইট" (light) generate করবে:

Attention weights: $h1=0.05$, $h2=0.05$, $h3=0.10$, $h4=0.80$

অর্থাৎ "lights" ($h4$) এর দিকে সবচেয়ে বেশি মনোযোগ!

$\text{Context_Vector_dynamic} = (0.05 \times h1) + (0.05 \times h2) + (0.10 \times h3) + (0.80 \times h4)$

Decoder যখন "বন্ধ" (off) generate করবে:

Attention weights: $h1=0.30$, $h2=0.55$, $h3=0.10$, $h4=0.05$

$\text{Context_Vector_dynamic} = (0.30 \times h1) + (0.55 \times h2) + (0.10 \times h3) + (0.05 \times h4)$

প্রতিটি time step-এ আলাদা Context Vector – Dynamic!

Attention Mechanism দীর্ঘ sentence-এর translation quality উন্নত করল। কিন্তু একটি বড় সমস্যা তখনও ছিল —

⚠ তখনও যে সমস্যা ছিল

Architecture-এ তখনও LSTM ছিল। LSTM sequential manner-এ data accept করে — একবারে একটি word। তাই **Parallel Processing** সম্ভব নয় → Training slow → বড় dataset-এ train করা যায় না → Transfer Learning-এর সুযোগ নেই → প্রতিটি নতুন project-এ scratch থেকে train করতে হয় → প্রচুর সময়, data ও অর্থ।

Chapter 3: "Attention is All You Need" (2017)

LANDMARK PAPER

"Attention is All You Need" — Vaswani et al. (Google Brain, 2017) — এই paper-ই Transformer Architecture introduce করে এবং পূর্ববর্তী সমস্ত সমস্যার সমাধান করে।

এই paper-টি এতটাই groundbreaking যে মনে হয় ভবিষ্যৎ থেকে কেউ এসে ২০১৭ সালে এটি লিখে গেছেন। এটি incremental পরিবর্তন ছিল না — এটি সম্পূর্ণ নতুন একটি Architecture।

তিনটি মূল উদ্ভাবন

১. **LSTM** বাদ দেওয়া: LSTM-এর পরিবর্তে সম্পূর্ণ Self-Attention ব্যবহার করা হয়, ফলে Parallel Processing সম্ভব হয়।

২. **হিতিশীল Architecture**: Residual Connections, Layer Normalization, Multi-Head Attention — এই components একসাথে একটি অত্যন্ত stable ও powerful architecture তৈরি করে।

৩. **Robust Hyperparameters**: ২০১৭ সালে যে hyperparameter values ব্যবহার হয়েছিল, সেগুলো আজও কার্যকর ও reliable।

২০০০–২০১৪

**RNN ও LSTM-এর যুগ**

NLP-তে RNN ও LSTM dominant ছিল। Statistical ML methods যেমন Naive Bayes, HMM, SVM, Bag-of-Words, N-gram ব্যবহার হতো। Word Embeddings-এর ধারণা (Word2Vec) এই সময়েই আসে।

২০১৪

**Encoder-Decoder + Attention Mechanism**

Sutskever-এর Seq2Seq paper এবং Bahdanau-এর Attention paper। Machine Translation-এর জন্য প্রথম কার্যকর deep learning solution।

২০১৭

**"Attention is All You Need" → Transformer জন্ম**

Google Brain-এর এই paper Transformer Architecture introduce করে। LSTM বাদ দিয়ে সম্পূর্ণ Attention-based approach — AI history-র সবচেয়ে গুরুত্বপূর্ণ মুহূর্তগুলোর একটি।

২০১৮

**BERT ও GPT-1 — Transfer Learning শুরু**

বিশাল dataset-এ Transformer train করা হয় এবং public domain-এ release করা হয়। NLP-তে Transfer Learning-এর যুগ শুরু হয়। Fine-tuning এর মাধ্যমে যেকোনো state-of-the-art result অর্জন করতে পারে।

২০১৮–২০২০

**বিভিন্ন Domain-এ Transformer প্রসার**

Vision Transformer (ViT) নিয়ে কাজ শুরু। DeepMind-এর AlphaFold 2 — protein structure prediction-এ বিপ্লব (Transformer ব্যবহার করে)। Science ও Biology research-এ Transformer।

২০২১



Generative AI-এর যুগ শুরু

GPT-3, DALL-E, Codex-এর আবির্ভাব। Text দিয়ে image তৈরি, code generation — AI এখন সাধারণ মানুষের কাছে পৌঁছে যায়। Midjourney, Stable Diffusion।

২০২২-এখন



ChatGPT ও AI Revolution

ChatGPT বিশ্বের সবচেয়ে দ্রুত বর্ধনশীল software হয়। Stable Diffusion, GPT-4, Claude, Gemini। AI-এর ব্যাপক সামাজিক প্রভাব। অসংখ্য startup ও লক্ষ কোটি ডলারের investment।

Transformer Architecture দেখতে জটিল মনে হলেও, এটিকে সহজে বোঝা সম্ভব। মূলত এটি একটি Encoder-Decoder Architecture যেখানে LSTM-এর পরিবর্তে Self-Attention ব্যবহার হয়।

| Self-Attention — Transformer-এর হৃদয়



ANALOGY

মনে করো একটি ক্লাসরুমে সব ছাত্র একে অপরের দিকে তাকাতে পারে। প্রতিটি ছাত্র নিজের উত্তর দেওয়ার সময় বাকি সবার দিকে তাকিয়ে সিদ্ধান্ত নেয় কে কতটা প্রাসঙ্গিক। এটাই *Self-Attention* — প্রতিটি *word* বাকি সব *word*-এর সাথে নিজের সম্পর্ক বিচার করে।



```
# Self-Attention এর মূল ধারণা:
# প্রতিটি word থেকে তিনটি vector তৈরি হয়:
#   Q = Query (আমি কী খুঁজছি?)
#   K = Key   (আমার কাছে কী আছে?)
#   V = Value (আমি কী দিতে পারি?)

import numpy as np

def self_attention(Q, K, V, d_k):
    """
    Q: Query matrix    (seq_len, d_k)
    K: Key matrix      (seq_len, d_k)
    V: Value matrix    (seq_len, d_v)
    d_k: dimension of key vectors
    """
    # Step 1: Q ও K-এর মধ্যে dot product (similarity score)
    scores = np.matmul(Q, K.T) / np.sqrt(d_k)

    # Step 2: Softmax দিয়ে probability তে convert
    # (প্রতিটি word কতটা important সেটা নির্ধারণ করে)
    attention_weights = np.exp(scores) / np.sum(np.exp(scores), axis=-1, keepdims=True)

    # Step 3: Value-এর weighted sum নীও
    output = np.matmul(attention_weights, V)
    return output, attention_weights

# উদাহরণ: "The cat sat on the mat"
# "cat" word-এর জন্য "sat" শব্দটি বেশি attention পাবে
# কারণ cat-ই sat করেছে!
```

Architecture-এর মূল Components

COMPONENT	কী করে	কেন দরকার
Multi-Head Self-Attention	একসাথে কয়েকটি ভিন্ন ভিন্ন perspective থেকে words-এর সম্পর্ক বিশ্লেষণ	Parallel processing সম্ভব করে; দীর্ঘ-দূরত্বের dependency ধরতে পারে

COMPONENT	কী করে	কেন দরকার
Positional Encoding	প্রতিটি word-এর position-এর তথ্য model-কে দেওয়া	Self-Attention নিজে থেকে position বোঝে না; এটি সেই অভাব পূরণ করে
Feed-Forward Neural Network	প্রতিটি position-এ independently non-linear transformation	Model-এর expressive power বাড়ায়
Residual Connection	Input-কে output-এর সাথে যোগ করে (skip connection)	Gradient vanishing সমস্যা দূর করে; deep network train করা সহজ হয়
Layer Normalization	প্রতিটি layer-এর output-কে normalize করে	Training stable রাখে; দ্রুত converge হয়
Cross-Attention (Decoder)	Decoder, Encoder-এর output-এর দিকে attention দেয়	Decoder বুঝতে পারে input sentence-এর কোন অংশ এখন relevant

Encoder vs Decoder — পার্থক্য

◆ Encoder-Only Models (যেমন: BERT)

পুরো input একসাথে দেখতে পারে (Bidirectional)।
Text classification, sentiment analysis, question answering-এ ভালো।

◆ Decoder-Only Models (যেমন: GPT)

শুধু আগের words দেখতে পারে (Unidirectional)।
Text generation-এ ব্যবহার হয়। ChatGPT এই ধরনের।

✓ ADVANTAGES

- ✓ **Scalability:** LSTM নেই, তাই Parallel Training সম্ভব — বড় dataset-এ দ্রুত train হয়
- ✓ **Transfer Learning:** Pre-train করে Fine-tune করা যায় — কম data ও সময়ে অসাধারণ result
- ✓ **Multimodal:** Text, image, speech, video — সব ধরনের data নিয়ে কাজ করতে পারে
- ✓ **Flexible Architecture:** Encoder-only, Decoder-only, বা উভয় — প্রয়োজন অনুযায়ী modify করা যায়
- ✓ **Vibrant Ecosystem:** HuggingFace, PyTorch-এর মতো library; অগণিত pre-trained model
- ✓ **Integration-friendly:** GANs, RL, CNN — অন্যান্য AI technique-এর সাথে সহজে combine করা যায়
- ✓ **Long-range Dependency:** Sentence-এর যেকোনো দূরত্বের words-এর সম্পর্ক ধরতে পারে

✗ DISADVANTAGES

- ✗ **High Compute:** Parallel processing-এর জন্য GPU দরকার — ব্যয়বহুল
- ✗ **Data Hungry:** ভালো result পেতে প্রচুর data দরকার
- ✗ **Energy Consumption:** বড় model train করতে প্রচুর electricity লাগে — পরিবেশগত উদ্বেগ
- ✗ **Black Box:** ভেতরে কী হচ্ছে বোঝা কঠিন — interpretability সমস্যা
- ✗ **Overfitting Risk:** প্রচুর parameters — data variety কম থাকলে overfit হয়
- ✗ **Ethical Concerns:** Training data-তে bias থাকলে model-ও biased হয়; copyright সমস্যা

| Scalability কেন গুরুত্বপূর্ণ — Transfer Learning-এর সাথে সম্পর্ক

এই chain of reasoning-টি বোঝা জরুরি:

🔗 CAUSAL CHAIN

Self-Attention → Parallel Processing সম্ভব →

Training দ্রুত হয় → বিশাল dataset-এ train করা যায় →

BERT, GPT-এর মতো Pre-trained Model তৈরি হয় →

Fine-tuning-এর মাধ্যমে যেকোনো ব্যবহার করতে পারে →

AI Democratized হয় → **AI Revolution**

সমস্যা	বিস্তারিত	সম্ভাব্য সমাধান
High Computational Cost	Parallel processing-এর জন্য শক্তিশালী GPU দরকার। GPT-4 train করতে কোটি কোটি ডলার খরচ হয়।	Pruning, Quantization, Knowledge Distillation — model ছোট করার techniques
Data Requirements	Domain-specific task-এ ভালো performance পেতে প্রচুর labeled data লাগে।	Transfer Learning ও Unsupervised Pre-training এই সমস্যা অনেকটা কমিয়েছে
Energy Consumption	Large Language Model train করতে একটি শহরের এক বছরের বিদ্যুৎ সমতুল্য energy লাগতে পারে।	Efficient architecture, green computing — research চলছে
Interpretability (Black Box)	Model কেন একটি নির্দিষ্ট উত্তর দিল সেটা ব্যাখ্যা করা কঠিন — Banking ও Healthcare-এ সমস্যা।	Explainable AI (XAI) research — attention visualization, SHAP values
Bias	Internet-এর data থেকে train হওয়ায় ঐ data-তে যে bias ছিল সেটা model-এও আসে।	Careful data curation, bias detection, RLHF (Reinforcement Learning from Human Feedback)
Ethical & Legal Concerns	অন্যের copyrighted data ব্যবহার করে train — বহু company-র বিরুদ্ধে lawsuit চলছে।	Consent-based data collection, licensing frameworks

**ChatGPT (OpenAI)**

GPT-3.5/4-এর উপর নির্মিত Chatbot। Generative Pre-trained Transformer। Text generation, coding, analysis — সব করতে পারে। আজকের সবচেয়ে জনপ্রিয় AI tool।

**DALL-E 2 (OpenAI)**

Text prompt থেকে photorealistic image generate করে। riding a horse in photorealistic style। Transformer-based।

**AlphaFold 2 (DeepMind)**

Protein structure prediction-এ বিপ্লব। প্রকৃতিতে বিদ্যমান প্রায় সব protein-এর 3D structure predict করেছে। Biology research-এ যুগান্তকারী।

**OpenAI Codex / GitHub Copilot**

Natural language থেকে code generate করে। integrate। Code type করতে কমান্ড দিয়ে।

**Runway ML, Stable Diffusion, Midjourney**

Text থেকে video (Runway ML), text থেকে high-quality image (Stable Diffusion, Midjourney)। Adobe Photoshop-এ integrate।

**Efficiency উন্নতি (Pruning, Quantization, Knowledge Distillation)**

Model-এর size কমিয়ে performance অক্ষুণ্ণ রাখার চেষ্টা। ছোট device-এও Transformer চালানো সম্ভব করা। এই তিনটি technique নিয়ে বর্তমানে প্রচুর গবেষণা হচ্ছে।

**Multimodal Capability-র বিস্তার**

Text, image, speech-এর পাশাপাশি sensory data, biometric data (fingerprint, retina scan), time-series data — সব ধরনের input handle করার সক্ষমতা বাড়ানো হবে।

**Domain-Specific Transformers**

ChatGPT-এর পাশাপাশি LegalGPT, DoctorGPT, TeacherGPT — domain-specific model যারা নিজের ক্ষেত্রে expert। একজন আইনি পরামর্শ চাইলে সে LegalGPT-এর কাছে যাবে।

**Multilingual Transformers**

বাংলা, হিন্দি, অন্যান্য আঞ্চলিক ভাষায় Transformer train করার কাজ চলছে। ভারতে Ola-র founder Krutrim AI-এর মতো company হিন্দিতে Transformer তৈরি করছে।

**Interpretability (White Box বানানো)**

Transformer-এর "black box" সমস্যা সমাধানের চেষ্টা। Model কেন কোনো সিদ্ধান্ত নিল সেটা ব্যাখ্যা করতে পারলে Banking ও Healthcare-এ Transformer-এর প্রবেশ সহজ হবে।

**Responsible AI Development**

Bias সম্পূর্ণ eliminate করা এবং ethical concerns overcome করা। Data collection-এ consent, licensing framework তৈরি।

HuggingFace Transformers library ব্যবহার করে মাত্র কয়েক লাইনের কোডে state-of-the-art NLP tasks করা যায়।

Installation



TERMINAL

```
pip install transformers torch
```

উদাহরণ ১: Sentiment Analysis (Fine-tuned BERT)



```
from transformers import pipeline

# মাত্র ৩ লাইনে একটি state-of-the-art Sentiment Analyzer!
# HuggingFace থেকে pre-trained BERT-based model download হবে
classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

# Test করা
texts = [
    "I absolutely love this product! It's amazing!",
    "This is the worst thing I have ever bought.",
    "The service was okay, nothing special."
]

results = classifier(texts)

for text, result in zip(texts, results):
    print(f"Text: {text[:50]}...")
    print(f"Label: {result['label']}, Confidence: {result['score']:.4f}")
    print()

# Output:
# Label: POSITIVE, Confidence: 0.9998
# Label: NEGATIVE, Confidence: 0.9997
# Label: NEGATIVE, Confidence: 0.5834
```

উদাহরণ ২: Text Generation (GPT-2)



```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

# GPT-2 model ও tokenizer load করো
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
model.eval()

# Prompt দাও
prompt = "Artificial intelligence is transforming the world by"
inputs = tokenizer.encode(prompt, return_tensors="pt")

# Text generate করো
with torch.no_grad():
    outputs = model.generate(
        inputs,
        max_length=100,
        num_return_sequences=1,
        temperature=0.8, # creativity নিয়ন্ত্রণ করে
        top_p=0.9, # nucleus sampling
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id
    )

# Output decode করো
generated = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(generated)
```

উদাহরণ ৩: Machine Translation (Helsinki-NLP)



```
from transformers import pipeline

# English to Bengali translation
# Helsinki-NLP এর pre-trained translation Transformer
translator = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-en-ROMANCE"
)

# ইংরেজি text কে French-এ অনুবাদ করো
english_text = "Transformers are revolutionizing artificial intelligence."
result = translator(english_text, max_length=200)

print(f"Original: {english_text}")
print(f"Translated: {result[0]['translation_text']}")
```

উদাহরণ 8: Fine-Tuning — নিজের Task-এ BERT Train করা



```
from transformers import (
    BertForSequenceClassification,
    BertTokenizer,
    TrainingArguments,
    Trainer
)
from torch.utils.data import Dataset
import torch

# ১. Custom Dataset তৈরি করো
class TextDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=128):
        self.encodings = tokenizer(
            texts,
            truncation=True,
            padding=True,
            max_length=max_length,
            return_tensors="pt"
        )
        self.labels = torch.tensor(labels)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        item = {k: v[idx] for k, v in self.encodings.items()}
        item['labels'] = self.labels[idx]
        return item

# ২. Pre-trained BERT load করো (Transfer Learning!)
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2 # Binary classification (positive/negative)
)

# ৩. তোমার নিজের ছোট dataset দাও
train_texts = ["Great product!", "Terrible service.", "I love it!"]
train_labels = [1, 0, 1] # 1=positive, 0=negative
train_dataset = TextDataset(train_texts, train_labels, tokenizer)
```



```
# ৪. Training Arguments
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=8,
    learning_rate=2e-5, # Fine-tuning-এ ছোট learning rate
    logging_steps=10,
)

# ৫. Fine-Tune করো!
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
)
trainer.train()

# BERT ইতিমধ্যে ইন্টারনেটের বিশাল text থেকে শিখেছে
# তোমার মাত্র কিছু example দিয়ে সেটাকে নিজের কাজে লাগাও!
# এটাই Transfer Learning-এর জাদু!
```

💡 মূল শিক্ষা

HuggingFace-এর `pipeline()` function দিয়ে মাত্র ২-৩ লাইনে state-of-the-art NLP task করা যায়। Fine-tuning-এর মাধ্যমে নিজের specific domain-এ BERT/GPT-কে adapt করা যায়। এটাই Transformer-এর democratization — ছোট startup বা individual researcher-ও এখন AI-powered product বানাতে পারে।



Quick Reference — সব এক জায়গায়

বিষয়	মূল তথ্য
Transformer কী?	Sequence-to-Sequence task-এর জন্য Neural Network Architecture (2017)
মূল Paper	"Attention is All You Need" — Vaswani et al. (Google Brain, 2017)
LSTM থেকে পার্থক্য	Self-Attention ব্যবহার করে → Parallel Processing → Scalable
Key Components	Self-Attention, Positional Encoding, Feed-Forward NN, Residual Connection, Layer Norm
বিখ্যাত Models	BERT (Encoder-only), GPT (Decoder-only), T5 (Both)
জনপ্রিয় Applications	ChatGPT, DALL-E, AlphaFold, GitHub Copilot, Midjourney
সেরা Library	HuggingFace Transformers (Python)
Transfer Learning	Pre-train on huge data → Fine-tune on small specific data
প্রধান সুবিধা	Scalability, Transfer Learning, Multimodal, Flexible
প্রধান সীমাবদ্ধতা	High compute cost, Data hungry, Black box, Bias